Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini
alex.santini2@studio.unibo.it

April 23, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for managing and replicating smart-city sensor data across multiple nodes, without any central coordinator. Each node ingests local readings through a pluggable sensor interface, maintains a local replica of global state, and exchanges updates with peers through push-pull gossip.

The system is not limited to synthetic data generation: in the demo setup, sensors are simulated for reproducibility, but the architecture is designed to connect real sensors as soon as their interface adapter is provided. Convergence follows an eventually consistent model with deterministic timestamp-based Last-Writer-Wins merging.

To remain operational in realistic distributed environments, the design combines gossip-based membership, adaptive failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while guaranteeing convergence after communication is restored.

Fault-injection experiments on multi-node deployments show stable recovery and consistent replica alignment after failures and partitions. The result is a robust foundation for decentralized smart-city monitoring, where data collection points are naturally distributed and continuity under partial failures is essential.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. The project implements a *distributed peer-to-peer application* for sensor *management and replication* in smart-city scenarios. Concretely, it combines:

- a node runtime service (Python process) handling sensor ingestion, replication, membership, failure detection, and protocol logic;
- inter-node TCP-based communication services (custom protocol);
- an optional observability layer for real-time monitoring.

Use case description. The software manages a cluster of sensor nodes that ingest readings and disseminate updates to peers, so that each node converges toward a coherent global view (eventual consistency with deterministic LWW conflict resolution).

In the current demo, sensor streams are simulated; however, the same ingestion interface allows integration of real sensor sources with no architectural changes to replication logic.

Typical users are smart-city monitoring stakeholders with two main human interaction profiles:

- *observational interaction* (control-room operators): inspect live sensor values and cluster status;
- *operational interaction* (technical operators): start/stop nodes, apply configuration changes, and run recovery or fault-handling procedures.

Machine-to-machine interaction is *continuous and periodic*: heartbeat exchange, gossip rounds, push-pull synchronization, and recovery synchronization after outages.

Interaction channels and devices:

- terminal/CLI for deployment and configuration;
- monitoring clients and automated tests for system observability;
- desktop/laptop web browser for optional dashboard-based monitoring.

Data handling and roles:

- nodes store replicated sensor records, membership/liveness metadata, topology snapshots, and introspection counters/events (node-local, runtime state);
- optional log files provide operational traces;
- main roles include: *node process* (autonomous participant), *cluster operator/developer*, and *observer client* (UI/tests/tools).

Why distribution is needed. Distribution is a core requirement, not an implementation detail:

- *fault tolerance and resilience*: no single coordinator; nodes can continue under crashes/restarts and temporary partitions;
- *eventual convergence at scale*: each node contributes updates and receives others through push/pull plus gossip dissemination;
- *resource and workload sharing*: sensing, merge, and communication work are spread across peers;
- *realistic decentralized environments*: smart-city infrastructures are geographically distributed by nature, so data is produced at the edge and synchronized across independent participants.

Overall, the project targets robust decentralized operation with observable, testable behavior under partial failures and dynamic cluster conditions.

2 Requirements Elicitation and Analysis

- The requirements must explain **what** (not how) the software being produced should do.
  - you should not focus on the particular problems, but exclusively on what you want the application to do.
- Requirements must be clearly identified, and possibly numbered
- Requirements are divided into:
  - **Functional**: some functionality the software should provide to the user
  - **Non-functional**: requirements that do not directly concern behavioural aspects, such as consistency, availability, etc.
  - **Implementation**: constrain the entire phase of system realization, for instance by requiring the use of a specific programming language and/or a specific software tool

        * these constraints should be adequately justified by political / economic / administrative reasons...
        * ... otherwise, implementation choices should emerge *as a consequence of* design

- If there are domain-specific terms, these should be explained in a glossary
- Each requirement must have its own **acceptance criterion**
  - these will be important for the validation phase

2.1 Relevant Distributed System Features

Motivate which distributed system features are relevant for your project, and which are not.

- transparency
  - does your system need to hide distribution details from users or developers?
  - is it important that failures, location, or replication are invisible?
- fault tolerance, dependability – availability, reliability, integrity, maintainability, safety
  - what happens if a component fails? is uninterrupted service required?
  - is data loss or corruption unacceptable?
  - how quickly must the system recover from faults?
- scalability
  - will the system need to handle increasing numbers of users, requests, or data?
  - is it expected to grow over time?
- security, trust
  - is sensitive data being processed or stored?
  - are there multiple user roles with different permissions?
  - is authentication or authorization required?
- resource sharing
  - do multiple users or components need access to shared resources?
  - is coordination or synchronization needed?
- openness, interoperability, heterogeneity of components
  - Will your system interact with external systems or use components built with different technologies?
  - Is standardization or compatibility important?

- evolvability, maintainability
  - Will the system need to be updated or extended after deployment?
  - Is long-term maintenance a concern?
- performance, concurrency, computation / communication efficiency, bandwidth
  - Are there strict requirements on response time or throughput?
  - Will many operations happen in parallel?
  - Is network usage a concern?
- economy, costs
  - Are there budget constraints for development, deployment, or operation?
  - Is minimizing resource usage important?

3 Design

This chapter explains the strategies used to meet the requirements identified in the analysis. Ideally, the design should be the same, regardless of the technological choices made during the implementation phase.

> You can re-arrange the sections as you prefer, but all the sections must be present in the end

Important: try to motivate your design choices in relation to the requirements and features identified in section 2 and section 2.1.

3.1 Architecture

- Which architectural style?
  - why?

3.2 Infrastructure

- are there *infrastructural components* that need to be introduced? *how many*?
  - e.g. *clients*, *servers*, *load balancers*, *caches*, *databases*, *message brokers*, *queues*, *workers*, *proxies*, *firewalls*, *CDNs*, *etc.*
- how do components *distribute* over the network? *where*?
  - e.g. do servers / brokers / databases / etc. sit on the same machine? on the same network? on the same datacenter? on the same continent?
- how do components *find* each other?
  - how to *name* components?
  - e.g. DNS, *service discovery*, *load balancing*, *etc.*

Component diagrams are welcome here

3.3 Modelling

- which **domain entities** are there?
  - e.g. *users*, *products*, *orders*, *etc.*
- how do *domain entities* **map to** *infrastructural components*?
  - e.g. state of a video game on central server, while inputs/representations on clients
  - e.g. where to store messages in an IM app? for how long?
- which **domain events** are there?
  - e.g. *user registered*, *product added to cart*, *order placed*, *etc.*
- which sorts of **messages** are exchanged?
  - e.g. *commands*, *events*, *queries*, *etc.*
- what information does the **state** of the system comprehend
  - e.g. *users' data*, *products' data*, *orders' data*, *etc.*

Class diagram are welcome here

3.4 Interaction

- how do components *communicate*? *when*? *what*?
- *which* **interaction patterns** do they enact?

Sequence diagrams are welcome here

3.5 Behaviour

- how does *each* component **behave** individually (e.g. in *response* to *events* or messages)?
  - some components may be *stateful*, others *stateless*
- which components are in charge of updating the **state** of the system? *when*? *how*?

State diagrams are welcome here

3.6 Data and Consistency Issues

- Is there any data that needs to be stored?
  - *what* data? *where*? *why*?
- how should *persistent data* be **stored**?

  - e.g. relations, documents, key-value, graph, etc.
  - why?
- Which components perform queries on the database?
  - *when*? *which* queries? *why*?
  - concurrent read? concurrent write? why?
- Is there any data that needs to be shared between components?
  - *why*? *what* data?

3.7 Fault-Tolerance

- Is there any form of data **replication** / federation / sharing?
  - *why*? *how* does it work?
- Is there any **heart-beating**, **timeout**, **retry mechanism**?
  - *why*? *among* which components? *how* does it work?
- Is there any form of **error handling**?
  - *what* happens when a component fails? *why*? *how*?

3.8 Availability

- Is there any **caching** mechanism?
  - *where*? *why*?
- Is there any form of **load balancing**?
  - *where*? *why*?
- In case of **network partitioning**, how does the system behave?
  - *why*? *how*?

3.9 Security

- Is there any form of **authentication**?
  - *where*? *why*?
- Is there any form of **authorization**?
  - which sort of *access control*?
  - which sorts of users / *roles*? which *access rights*?
- Are **cryptographic schemas** being used?
  - e.g. token verification,
  - e.g. data encryption, etc.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

> If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
  - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
  - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
  - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
  - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
  - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components ***unit*-test**ed?
- how was communication, interaction, and/or integration among components tested?
- how to ***end-to-end*-test** the system?
  - e.g. production vs. test environment
- for each test specify:
  - rationale of individual tests
  - how were the test automated
  - how to run them
  - which requirement they are testing, if any

recall that *deployment* **automation** is commonly used to *test* the system in *production-like* environment

recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
  - what did you test?
  - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
  - provide instructions
  - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
  - provide instructions
  - provide expected outcomes
  - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References